

CRM Ambiental — Alcaldía de Envigado

Sistema de gestión de quejas ambientales basado en **EspoCRM + PostgreSQL**, con capa custom para radicación, asignación, visita de campo, comunicaciones, documentos PDF y tablero de control.

Dato	Valor
Repositorio	<code>github.com/Geotrends/CRMALCALDIA</code>
Rama principal	<code>main</code> (única rama de trabajo y despliegue)
Producción	Dokploy (rebuild de imagen Docker)
Versión de despliegue	Ver archivo <code>.deploy-version</code> en la raíz

Índice

- 1. Estructura del repositorio
- 2. Qué hay dentro de `espocrm-custom`
- 3. Requisitos previos
- 4. Ejecución local (Docker)
- 5. Desarrollo local (cambios sin rebuild)
- 6. Despliegue en Dokploy (producción)
- 7. Ramas y flujo de trabajo Git
- 8. Usuarios de prueba
- 9. Verificación y diagnóstico
- 10. Interfaz de usuario (tema institucional)
- 11. Documentación adicional

Estructura del repositorio

```
CRMALCALDIA/
├── espocrm-custom/      # Todo el código custom del CRM (backend + frontend)
├── scripts/            # Deploy, roles, permisos, migraciones y reparaciones
├── formatos/          # Plantillas Word/Excel oficiales (PDFs del sistema)
├── exports/           # Excel maestro (excelAlcaldia.xlsx) – catálogos
├── docker/            # Dockerfile y scripts de arranque del contenedor
├── sql/               # Esquema PostgreSQL de referencia (no es migración activa)
├── docs/              # Documentación operativa y de cumplimiento
├── backups/           # Plantillas de entorno (env.txt)
├── docker-compose.yml  # Stack producción (Dokploy y base local)
├── docker-compose.dev.yml # Override solo para desarrollo local
├── docker-compose.tools.yml # pgAdmin opcional (local)
├── .deploy-version     # Huella legible del último despliegue custom
├── .env.example        # Plantilla de variables de entorno
└── README.md          # Este archivo
```

Descripción de cada carpeta

Carpeta / archivo	Para qué sirve
<code>espocrm-custom/</code>	Extensión EspoCRM: PHP (hooks, servicios, controladores), metadata JSON (pantallas, campos, permisos), JavaScript/CSS (UI por rol), plantillas Handlebars y scripts Python para generar PDFs.
<code>scripts/</code>	Scripts PHP ejecutados en el deploy: creación de roles, permisos, dashboards, catálogos, migraciones legacy y reparaciones puntuales. Incluye <code>verify-custom-deploy.sh</code> para comprobar que el custom quedó bien aplicado.
<code>formatos/</code>	Documentos fuente: <code>FormatoSolicitud.doc</code> , <code>ActaVisita2.docx</code> , <code>ActuoArchivo.docx</code> , <code>ActaVisita.xlsx</code> . El sistema los usa para generar PDFs.
<code>exports/</code>	<code>excelAlcaldia.xlsx</code> — maestro de desplegables y exportación institucional.
<code>docker/</code>	<code>espocrm/Dockerfile</code> (imagen con LibreOffice + Python), <code>entrypoint-with-deploy.sh</code> (auto-deploy al arrancar), <code>run-custom-init.sh</code> (primer deploy tras instalación).
<code>sql/</code>	<code>esquema.sql</code> — volcado de referencia del esquema PostgreSQL. La fuente de verdad en runtime es la metadata de EspoCRM, no este archivo.
<code>docs/</code>	Consultas SQL de validación (<code>CONSULTAS-BD-VALIDACION.md</code>) y mapa de cumplimiento de objetivos (<code>ESTADO-CUMPLIMIENTO-OBJETIVOS.md</code>).
<code>backups/</code>	<code>despliegue-inicial/env.txt</code> — plantilla de variables para Dokploy o <code>.env</code> local.
<code>.deploy-version</code>	Texto corto (ej. <code>ui-doc-list-no-categoria-2026-07-02</code>) que el contenedor compara para saber si debe volver a aplicar el custom. Actualízalo en cada cambio relevante de frontend o metadata.

Qué hay dentro de `espocrm-custom`

```
espocrm-custom/
├── Controllers/           # APIs custom (timeline, cronograma, reportes...)
├── Entities/             # Entidades propias: ActaVisita, ActuoArchivo, ComunicacionCaso...
├── Hooks/                # Lógica al guardar/crear (radicado, notificaciones, PDFs...)
├── Tools/                # Servicios de negocio reutilizables (por dominio: CaseObj, Party...)
├── EntryPoints/          # Descarga de PDF/Excel (URLs directas)
├── Jobs/                 # Tareas programadas (alertas de vencimiento)
├── Classes/              # ACL, filtros de listado, RecordHooks
├── Resources/
│   ├── metadata/         # Definición de entidades, layouts, clientDefs, scopes
│   ├── i18n/             # Traducciones (es_ES principal)
│   └── layouts/          # Diseño de formularios y listas
└── files/
    ├── client/custom/    # Frontend: vistas, helpers, CSS, dashboard, .tpl
    │   ├── res/css/      # Hoja principal main.css + módulos 01-24
    │   └── src/loader/    # Scripts que cargan tema, toasts, i18n, navbar
    └── scripts/          # Python: relleno de formatos Word → PDF
```

Frontend — loaders registrados en `client.json`

Script	Función
<code>theme-navbar.js</code>	Navbar lateral, barra superior, CSS inline de respaldo (botones, búsqueda, campana)
<code>theme-buttons.js</code>	Botones/badge verde pastel; carga <code>22-ui-toasts.css</code> y <code>23-buttons.css</code>
<code>theme-login.js</code>	Pantalla de login (logo Envigado, botón verde pastel)
<code>ui-toasts.js</code>	Toasts y confirmaciones laterales (reemplaza banner superior)
<code>i18n-personas.js</code>	Traducciones de menú («Crear caso», etc.)
<code>notification-views.js</code>	Panel de notificaciones custom
<code>session-security.js</code>	Sesión y cierre de pestañas

Frontend — CSS principal (`files/client/custom/res/css/`)

Archivo	Contenido
<code>main.css</code>	Importa todos los módulos (punto de entrada en <code>cssList</code>)
<code>22-ui-toasts.css</code>	Toasts, confirmaciones, anuncios nativos EspoCRM
<code>23-buttons.css</code>	Botones píldora, badges de estado, búsquedas
<code>06-case.css</code>	Estados del caso con iconos
<code>09-navbar.css</code>	Navbar, campana con contador
<code>15-login.css</code>	Pantalla de inicio de sesión
<code>24-document-categories.css</code>	Iconos de categoría documental (detalle)

En runtime, todo esto se copia a `/var/www/html/custom/Espo/Custom` dentro del contenedor EspoCRM.

Requisitos previos

- **Docker y Docker Compose** (v2)
 - **Git**
 - Para desarrollo local: puertos libres **8080** (web), **8081** (websocket, opcional), **5433** (PostgreSQL)
-

Ejecución local (Docker)

1. Clonar el repositorio

```
git clone https://github.com/Geotrends/CRMALCALDIA.git
cd CRMALCALDIA
git checkout main
```

2. Configurar variables de entorno

```
cp .env.example .env
# o: cp backups/despliegue-inicial/env.txt .env
```

Edita `.env` y cambia al menos:

- `POSTGRES_PASSWORD`
- `ESPOCRM_ADMIN_PASSWORD`
- `ESPOCRM_SITE_URL=http://localhost:8080`

Importante: el archivo `.env` no se sube a GitHub (está en `.gitignore`).

3. Levantar el stack (con volúmenes de desarrollo)

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build
```

Esto levanta:

Servicio	Función
espocrm-db	PostgreSQL 16
espocrm	Apache + EspoCRM + auto-deploy
espocrm-init	Primer deploy del custom (una vez)
espocrm-daemon	Cola de trabajos EspoCRM
espocrm-websocket	Notificaciones en tiempo real (puerto 8081)

4. Acceder

- **CRM:** `http://localhost:8080`
- **Admin inicial:** usuario y contraseña definidos en `.env` (`ESPOCRM_ADMIN_USERNAME` / `ESPOCRM_ADMIN_PASSWORD`)

La primera vez puede tardar varios minutos (instalación + deploy de roles y catálogos).

5. Acceder a la base de datos con pgAdmin (local)

pgAdmin es **solo para desarrollo local**. No forma parte del despliegue en Dokploy.

Levantar pgAdmin

Con el stack ya en marcha (paso 3), añade pgAdmin:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml -f docker-compose.tools.yml up -d --pgadmin
```

Si aún no tienes el CRM arriba, puedes levantar todo junto:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml -f docker-compose.tools.yml up -d --build
```

Abrir la interfaz web

1. En el navegador entra a: **`http://localhost:5050`**
(o el puerto que hayas puesto en `PGADMIN_PORT` dentro de `.env`).
2. Inicia sesión con las credenciales de pgAdmin de tu `.env` :

Campo	Variable en <code>.env</code>	Valor por defecto
Email	<code>PGADMIN_DEFAULT_EMAIL</code>	<code>admin@local.test</code>
Contraseña	<code>PGADMIN_DEFAULT_PASSWORD</code>	<code>cambiar_pgadmin</code>

Conectar el servidor PostgreSQL (primera vez)

1. En el panel izquierdo, clic derecho en **Servers** → **Register** → **Server...**

2. Pestaña **General**:

- **Name:** CRM Alcaldía (o el nombre que prefieras)

3. Pestaña **Connection**:

Campo	Valor
Host name/address	espocrm-db
Port	5432
Maintenance database	valor de POSTGRES_DB (por defecto espocrm)
Username	valor de POSTGRES_USER (por defecto espocrm)
Password	valor de POSTGRES_PASSWORD de tu .env
Save password?	Sí (recomendado en local)

1. Clic en **Save**.

Nota: usa el host `espocrm-db` y el puerto **5432** (red interna de Docker).

El puerto **5433** de tu `.env` (`POSTGRES_PORT`) es solo para conectar desde tu Mac con otra herramienta (DBeaver, `psql` , etc.), no desde pgAdmin dentro de Docker.

Navegar por las tablas

1. En el árbol izquierdo expande:
Servers → **CRM Alcaldía** → **Databases** → **espocrm** → **Schemas** → **public** → **Tables**
2. Clic derecho en una tabla → **View/Edit Data** → **All Rows** para ver registros.
3. Para ejecutar SQL: clic derecho en la base **espocrm** → **Query Tool**, pega una consulta (por ejemplo desde `docs/CONSULTAS-BD-VALIDACION.md`) y pulsa **F5** o el botón ►.

Si no conecta

- Verifica que `espocrm-db` esté corriendo: `docker compose ps`
- Revisa que `POSTGRES_PASSWORD` en `.env` coincida con la que pones en pgAdmin.
- Si cambiaste la contraseña después de crear el volumen de BD, puede hacer falta recrear el volumen (solo en local, borra datos):

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml down -v
docker compose -f docker-compose.yml -f docker-compose.dev.yml -f docker-compose.tools.yml up -d --build
```

Desarrollo local (cambios sin rebuild)

Con `docker-compose.dev.yml` , estas carpetas se montan en el contenedor:

- `espocrm-custom/`
- `scripts/`

- `formatos/`
- `exports/`

Tras cambiar código custom, aplica el deploy manual:

```
bash scripts/deploy-custom.sh
```

Ese script copia el custom al contenedor, ejecuta rebuild de EspoCRM, corre los pasos de configuración (roles, permisos, etc.) y limpia caché.

Para ver logs:

```
docker compose logs -f espocrm
```

Para detener:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml down
```

Despliegue en Dokploy (producción)

Rama y repositorio

- **Rama:** `main`
- Dokploy debe apuntar a `main` del repo `Geotrends/CRMALCALDIA`.

No hay otra rama de despliegue: todo lo que se mergea o pushea a `main` es lo que va a producción.

Configuración en Dokploy (una sola vez)

Opción	Valor
Build type	Dockerfile
Dockerfile path	<code>docker/espocrm/Dockerfile</code>
Compose file	<code>docker-compose.yml</code>
Rebuild on deploy	Activado (obligatorio)

Variables de entorno (Dokploy → Environment)

Copia desde `backups/despliegue-inicial/env.txt` y ajusta:

Obligatorias:

```
POSTGRES_DB=espocrm
POSTGRES_USER=espocrm
POSTGRES_PASSWORD=...
POSTGRES_PORT=5432

ESPOCRM_ADMIN_USERNAME=admin
ESPOCRM_ADMIN_PASSWORD=...
ESPOCRM_SITE_URL=https://tu-dominio.ejemplo.com
ESPOCRM_TIME_ZONE=America/Bogota
```

Opcionales:

- `DEPLOY_VERSION` — si Dokploy no inyecta commit hash y quieres forzar redeploy.
- `ESPOCRM_WEB_SOCKET_URL` — URL `wss://tu-dominio/ws` si usas websocket en producción.
- Variables SMTP — para correo saliente (aún no probado de punta a punta).

`ESPO_RUN_AUTO_DEPLOY=1` ya viene en la imagen; no hace falta definirlo en Dokploy.

Flujo habitual de publicación

1. Desarrollas y pruebas en local (opcional)
2. `git add ...` && `git commit ...` && `git push origin main`
3. Dokploy → Redeploy (con rebuild de imagen, no solo restart)
4. En logs del servicio `espocrm` debe aparecer: `==> Auto-deploy completado.`
5. En el navegador: `Ctrl+Shift+R` (recarga forzada)

Qué hace el auto-deploy al arrancar

1. Compara `.deploy-version` de la imagen con la versión ya aplicada en el servidor.
2. Si hay cambio, ejecuta `scripts/deploy-custom-dokploy.sh` :
 - Copia `espocrm-custom/` al contenedor
 - Rebuild de EspoCRM
 - Ejecuta scripts de roles, permisos, catálogos y migraciones legacy (si aplican)
 - Copia plantillas desde `formatos/` y Excel desde `exports/`

Si los cambios no se ven en producción

1. Confirmar que el **push** llegó a `main` en GitHub.
2. En Dokploy, usar **Redeploy con rebuild** (no solo restart).
3. Revisar logs del contenedor `espocrm` al inicio.
4. Si aparece `Auto-deploy: sin cambios (omitido)`, forzar rebuild o actualizar `.deploy-version` en el repo.
5. Ejecutar verificación manual (dentro del servidor):

```
docker exec espocrm bash /opt/bootstrap/repo/scripts/verify-custom-deploy.sh
```


Ramas y flujo de trabajo Git

Rama	Uso
<code>main</code>	Rama única de integración y producción. Todo el equipo trabaja aquí o mergea aquí antes de Dokploy.
Tags (opcional)	Para hitos de cliente: <code>git tag v1.0-demo && git push origin v1.0-demo</code>

Recomendación práctica:

```
# Antes de empezar
git pull origin main

# Después de tus cambios
git add .
git commit -m "Descripción clara del cambio"
git push origin main

# Actualizar .deploy-version cuando el cambio afecte custom en producción
echo "ui-mi-cambio-$(date +%Y-%m-%d)" > .deploy-version
```

No uses ramas largas paralelas sin merge a `main` : Dokploy solo despliega lo que esté en `main` .

Usuarios de prueba

Tras el deploy, el script `seed-alcaldia-roles.php` crea roles y usuarios de prueba. Entra con **un rol a la vez** (cerrar sesión entre pruebas):

Rol	Usuario	Contraseña
Inspección	<code>inspeccion</code>	<code>inspeccion2026</code>
Radicación	<code>radicacion</code>	<code>radicacion2026</code>
Asignación	<code>asignacion</code>	<code>asignacion2026</code>
Patrullaje	<code>patrullaje</code>	<code>patrullaje2026</code>

Orden sugerido de prueba del flujo:

Inspección crea caso → Radicación radica → Asignación asigna patrullero → Patrullaje diligencia acta → Inspección revisa y cierra.

Para demos al cliente, preferir usuarios de rol (no `admin`).

Verificación y diagnóstico

Comando	Cuándo usarlo
<code>bash scripts/verify-custom-deploy.sh</code>	Tras deploy local (<code>docker exec espocrm bash ...</code> en producción)
<code>docker exec espocrm php /opt/bootstrap/repo/scripts/roles/configure-role-inspeccion.php</code>	Reparar permisos del rol Inspección (solo dentro del contenedor)
<code>docker compose logs espocrm</code>	Errores de arranque o auto-deploy
<code>docker compose ps</code>	Estado de contenedores

El script de verificación comprueba, entre otras cosas: acciones custom del Case, kanban, timeline, colores y archivos clave del frontend.

Checklist visual post-deploy (UI)

Tras **Redeploy + Ctrl+Shift+R**, revisar en el navegador:

Elemento	Esperado
Botón Crear caso	Verde pastel, forma píldora
Búsquedas (lista y navbar)	Ovaladas, fondo blanco
Estados en lista/detalle	Píldora con icono (no rombo)
Campana notificaciones	Número visible, badge verde pastel
Guardar registro	Toast derecho con spinner («Guardando...»)
Guardado exitoso	Toast con check verde
Login	Botón «Iniciar sesión» verde pastel
Lista Documentos (plantillas)	Sin columna Categoría

Si la UI no cambia: confirmar push a `main`, rebuild en Dokploy, `.deploy-version` actualizado y que `client.json` incluya `theme-navbar.js`, `theme-buttons.js` y `ui-toasts.js` en `scriptList`.

Interfaz de usuario (tema institucional)

La capa visual es **solo presentación**; no modifica permisos ni flujos de negocio.

- **Colores:** verde pastel institucional (`#eefaf5` → `#d8f3e8` , texto `#1a5c47`).
- **Toasts:** esquina superior derecha; icono alusivo al mensaje (carga, éxito, error, info).
- **Producción:** además de `cssList` , varios estilos se inyectan por JavaScript para evitar caché obsoleto (`theme-navbar.js` , `theme-buttons.js`).
- **Metadata UI:** layouts en `Resources/layouts/` ; cambios de columnas (ej. quitar Categoría en `Document/list.json`) requieren rebuild/clear-cache de EspoCRM.

Detalle de cumplimiento por objetivo del proyecto: docs/ESTADO-CUMPLIMIENTO-OBJETIVOS.md (sección *Complemento transversal — Interfaz de usuario institucional*).

Documentación adicional

Archivo	Contenido
docs/GUIA-HANDOFF-PROYECTO.md	Guía principal de handoff: estructura, reglas, flujos, deploy y checklist
docs/handoff/	Anexos A–H: inventarios exhaustivos (helpers, views, campos Case, hooks, backend, scripts, CSS, metadata)
docs/CONSULTAS-BD-VALIDACION.md	Consultas SQL para validar datos en PostgreSQL (Dokploy / pgAdmin)
docs/ESTADO-CUMPLIMIENTO-OBJETIVOS.md	Objetivos del proyecto vs. lo implementado en el CRM
backups/despliegue-inicial/env.txt	Plantilla de variables para Dokploy

Resumen rápido

Entorno	Comando / acción
Local — primera vez	<code>cp .env.example .env</code> → editar → <code>docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --build</code>
Local — aplicar cambios	<code>bash scripts/deploy-custom.sh</code>
Local — pgAdmin	<code>docker compose ... -f docker-compose.tools.yml up -d pgadmin</code> → <code>http://localhost:5050</code>
Producción	<code>git push origin main</code> → Dokploy Redeploy con rebuild → Ctrl+Shift+R en el navegador
Rama	Siempre <code>main</code>